PROCEDIMIENTO DE IMPLEMENTACIÓN DEL SOFTWARE

PROYECTO:
SOFTWARE PARA LA CREACIÓN DE LA APLICACIÓN "ALERTA MUJER"

AUTOR: FREINIER STEVEN CARDONA PEREZ

SERVICIO NACIONAL DE APRENDIZAJE –SENA
ANALISIS Y DESARROLLO
DE SOFTWARE – 3145555

2026

Resumen

Este procedimiento organiza la preparación y verificación local de Alerta Mujer como un sistema integrado de base de datos, servicio de aplicación, sitio web y aplicación móvil. Presenta tareas, responsables, comprobaciones y criterios de recuperación con un lenguaje útil para el equipo técnico y para quienes evalúan el producto.

Palabras clave: Alerta Mujer, software, documentación de producto, seguridad, apoyo.

Contenido

1. Propósito y alcance de la implementación
2. Personas responsables y condiciones de trabajo
3. Requisitos del equipo y preparación inicial
4. Configuración privada y manejo de credenciales
5. Inicio ordenado de la base de datos
6. Migraciones y administrador inicial
7. Comprobación de datos, roles y persistencia
8. Parámetros del servicio de aplicación
9. Ejecución local del servicio
10. Alternativa de ejecución del servicio con Docker
11. Preparación de la experiencia web
12. Preparación de la experiencia móvil
13. Contratos de comunicación y permisos
14. Recorrido de registro y sesión
15. Contactos y activación de una alerta
16. Conversación, evidencia y atención administrativa
17. Diagnóstico, seguimiento y evidencias de aceptación
18. Cierre, respaldo y recuperación del entorno

1. Propósito y alcance de la implementación

Alerta Mujer reúne una aplicación móvil para usuarias, un sitio web público con panel administrativo, un servicio central y una base de datos PostgreSQL. Implementar el producto significa preparar estos componentes en un orden que permita comprobar su interacción. La base conserva identidades, relaciones de contacto, emergencias, ubicaciones, mensajes y referencias de evidencia; el servicio aplica las reglas y los permisos; las interfaces muestran las acciones autorizadas a cada persona. Esta secuencia evita atribuir a la pantalla una operación que todavía no ha confirmado el servicio.

El procedimiento está pensado para un entorno local de integración. Las direcciones con localhost, 127.0.0.1 o una dirección de red del equipo son ejemplos de acceso dentro de ese entorno y deben ajustarse a la red utilizada. Una ejecución completa abarca el inicio de la base y sus cambios de estructura, la configuración del servicio, la conexión de las interfaces y un recorrido funcional de extremo a extremo (Equipo del proyecto Alerta Mujer, 2026a, 2026b, 2026c). La aceptación se apoya en resultados observables: inicio sin errores de configuración, respuestas autenticadas, alertas persistidas y consulta de datos autorizados. Cada comprobación tiene un resultado esperado y una acción de recuperación, de modo que una falla pueda localizarse sin comprometer datos ni alterar innecesariamente otros componentes.

Como criterio de inicio, anote la fecha, el equipo utilizado, el sistema operativo y las versiones principales de las herramientas. Esa ficha identifica el contexto de cada resultado y evita comparar una prueba de navegador con otra de teléfono como si fueran equivalentes. También conviene acordar una cuenta de prueba por rol y un contacto que acepte la invitación; sin esas identidades, la comprobación del SOS quedaría incompleta.

2. Personas responsables y condiciones de trabajo

La implementación requiere coordinación entre quien administra la base de datos, quien configura el servicio y quien prepara las interfaces. Una misma persona puede asumir varias tareas en una práctica académica, siempre que distinga qué credenciales utiliza en cada momento. El responsable de datos prepara las contraseñas locales, aplica las migraciones y verifica los permisos. El responsable del servicio confirma la conexión con la base, la clave de sesión, el correo para códigos de verificación y el almacenamiento privado de fotografías. Quien atiende las interfaces configura las direcciones de acceso y comprueba que la experiencia web y móvil se comuniquen con el servicio.

Antes de iniciar, conviene acordar un identificador de ejecución, las personas autorizadas a ver los registros de prueba y el momento para limpiar los datos sintéticos. Las cuentas utilizadas en la validación deben ser ficticias y los números o correos deben pertenecer a quienes realizan la prueba. No se deben compartir contraseñas por capturas de pantalla ni incorporar archivos de configuración privada a la documentación. El equipo registra qué pasos terminó, cuáles necesitaron corrección y qué evidencia observó. Este registro breve facilita que un docente o evaluador reconstruya el recorrido sin tener que leer detalles internos del programa.

Para no confundir responsabilidades, el equipo puede mantener una tabla de cuatro columnas: componente, responsable, dato de entrada y evidencia de salida. Por ejemplo, la salida de la preparación de datos es un historial de migraciones sin errores; la del servicio es un arranque válido; la de cada interfaz es una operación confirmada contra la API. Esta tabla ayuda a asignar una corrección a quien puede ejecutarla.

3. Requisitos del equipo y preparación inicial

El equipo local necesita Docker Desktop con Docker Compose v2 para PostgreSQL y Liquibase, PowerShell para los comandos indicados, Java 21 para el servicio y Node.js con npm para las interfaces. También requiere conexión a internet al obtener dependencias por primera vez, puertos disponibles para base, servicio y web, y una red que permita al teléfono alcanzar al equipo anfitrión cuando se use un dispositivo físico. Es prudente comprobar los puertos antes de iniciar, porque otro proceso puede ocuparlos y producir un error que parezca una falla de la aplicación.

La configuración de referencia publica PostgreSQL en 127.0.0.1:5434, el servicio en localhost:8081 y la web de desarrollo en localhost:5173. Dentro de la red de contenedores, la base escucha en postgres:5433. Estas direcciones cumplen funciones distintas: 127.0.0.1 identifica el propio equipo, mientras que postgres identifica el servicio de base de datos en Docker. Un teléfono físico necesita la dirección de red local del equipo para acceder al servicio; su propio localhost no apunta al ordenador. Antes de continuar, verifique que Docker responde, que Java muestra la versión 21 y que npm puede instalar paquetes. Si alguna condición falta, resuélvala primero para evitar diagnósticos confusos en pasos posteriores.

Revise además la memoria y el espacio disponible antes de crear contenedores y dependencias. Una descarga interrumpida puede dejar un error de instalación que no guarda relación con Alerta Mujer. Si se usa un emulador, inícielo antes de la prueba móvil y compruebe que puede abrir una página de red. Para el dispositivo físico, verifique desde su navegador una dirección local del equipo antes de atribuir una falla a la aplicación.

4. Configuración privada y manejo de credenciales

Cada ecosistema proporciona un ejemplo de configuración. En el equipo de trabajo se crea un archivo .env propio a partir de ese ejemplo y se reemplazan los valores de muestra por datos locales. La base necesita contraseñas para PostgreSQL y para los usuarios técnicos de migración y aplicación. El servicio necesita los datos de conexión, una clave de sesión de al menos 32 bytes, orígenes web permitidos, un lugar privado para evidencias y parámetros de correo para enviar códigos. Las interfaces necesitan conocer la dirección alcanzable del servicio. Estos valores se coordinan; una contraseña de aplicación distinta en base y servicio impide la conexión.

No conviene usar una sola credencial para todas las funciones. Liquibase entra con el usuario de migración y el servicio entra con el usuario de aplicación, cuyos permisos son más restringidos. El administrador inicial se define mediante parámetros específicos de la base cuando se habilita su creación. En esta guía se describen nombres y propósito de variables sin publicar valores secretos. La persona responsable debe conservar el archivo privado fuera del control de versiones y evitar copiarlo a informes académicos. Si una credencial se expone accidentalmente, debe sustituirse y actualizarse en los componentes que dependen de ella antes de repetir las pruebas.

Una forma sencilla de comprobar coherencia es comparar, sin mostrar sus valores, los nombres de las variables que comparten base y servicio. El usuario de aplicación, el nombre de la base y el puerto publicado deben formar una misma ruta de conexión. Para los secretos, confirme coincidencia mediante una prueba de acceso, no mediante una captura del archivo privado. Al rotar credenciales, anote qué componente se reinició y en qué orden.

5. Inicio ordenado de la base de datos

La base de datos se inicia antes que el servicio, porque este valida al arrancar que la estructura esperada ya existe. Desde la carpeta del ecosistema de base de datos, el equipo crea su configuración privada y revisa las variables de nombre, puertos, usuarios y contraseñas. La configuración de ejemplo utiliza alertamujer_db, publica el puerto 5434 y reserva el usuario alertamujer_app para la aplicación. El servicio de migraciones utiliza alertamujer_migrator. Si se modifica un puerto, el cambio debe reflejarse en la conexión del servicio.

```
Copy-Item .env.example .env
docker compose config --quiet
docker compose up -d
docker compose logs liquibase
```

El primer comando crea una copia local; se edita antes de iniciar los contenedores. La validación de Compose detecta variables obligatorias ausentes o una estructura incorrecta. Al iniciar, PostgreSQL supera su comprobación de salud y Liquibase aplica los cambios pendientes de forma ordenada. El registro de Liquibase debe mostrar que terminó sin error. docker compose up -d mantiene los servicios en segundo plano para continuar la integración. Si la primera ejecución falla, se revisan variables, puertos y salud de PostgreSQL antes de repetir; no se borra el volumen como reacción inicial, porque allí residen los datos persistentes.

Después del primer arranque, consulte docker compose ps para distinguir el contenedor persistente de PostgreSQL del proceso de Liquibase, que termina una vez aplicados los cambios. Un estado finalizado de Liquibase puede ser normal si el registro confirma éxito. Si los registros muestran un cambio rechazado, conserve el mensaje y compruebe el historial antes de volver a ejecutar. Repetir el comando sobre una base ya preparada no debe sustituir el diagnóstico de la causa.

6. Migraciones y administrador inicial

Liquibase administra la estructura de la base mediante cambios versionados. El servicio de aplicación no crea tablas ni ejecuta migraciones; al iniciar, comprueba que las tablas existentes correspondan a lo que necesita. Este reparto de tareas ayuda a mantener una secuencia auditable y evita que una ejecución de la aplicación cambie la estructura sin control. En una base nueva, la operación normal de Compose aplica los cambios pendientes una vez que PostgreSQL está disponible. También es posible validar el conjunto de cambios de manera explícita sin modificar los datos.

```
docker compose run --rm liquibase validate
docker compose run --rm liquibase status --verbose
```

La creación del administrador inicial depende de ADMIN_BOOTSTRAP_ENABLED y de los datos ADMIN_BOOTSTRAP_* del archivo privado de la base. Si se habilita, deben completarse todos los campos requeridos con una identidad y una contraseña controladas por el equipo responsable. La cuenta administrativa permite acceder al panel web y no debe usarse como cuenta corriente de una usuaria. Después de verificar su creación, el equipo debe custodiar la credencial conforme a sus reglas de acceso. Si se cambian contraseñas sobre un volumen de base ya existente, se usa el procedimiento de reconciliación de roles indicado por el ecosistema de datos; copiar un nuevo .env por sí solo no cambia las credenciales ya almacenadas en PostgreSQL.

El administrador inicial debe reservarse para la persona o equipo responsable del panel y no servir como cuenta de contacto en la prueba. Después de su creación, entre una vez desde la web para confirmar la credencial y cierre la sesión. Si el panel rechaza el acceso, diferencie un error de contraseña de un problema de conexión antes de reconfigurar la base. La secuencia de migración y bootstrap se consulta en los registros de Compose.

7. Comprobación de datos, roles y persistencia

Una base iniciada no está necesariamente lista para la aplicación. Primero se confirma que PostgreSQL acepta conexiones, que Liquibase completó la secuencia y que el usuario de aplicación dispone de las operaciones necesarias sin recibir permisos de estructura. El ecosistema de datos ofrece verificaciones de conexión, codificación UTF-8, zona horaria UTC y persistencia del volumen. También comprueba la separación entre el propietario técnico, el usuario de migración y el usuario utilizado por el servicio. Estas comprobaciones dan más información que observar únicamente si un contenedor aparece como activo.

```
docker compose ps
docker compose run --rm liquibase validate
docker compose run --rm liquibase history
```

Los resultados deben interpretarse en conjunto. Un error de autenticación apunta primero a las contraseñas y al estado del volumen; una migración incompleta apunta a Liquibase; un error de puerto apunta a Docker o a otro servicio local. La base conserva seis áreas activas de datos: configuración, identidad, perfil, contactos, emergencias y auditoría. Las fotografías se guardan como archivos privados fuera de PostgreSQL, mientras que la base conserva sus referencias y metadatos. Una verificación satisfactoria de datos habilita el siguiente paso, pero todavía no demuestra que la web o el móvil estén conectados; esa interacción se comprueba más adelante.

Para verificar persistencia, cree un dato sintético solo después de que el servicio esté conectado, detenga los contenedores sin eliminar volúmenes y confirme su presencia al reiniciar. Esta prueba se realiza con autorización y registra qué dato se usó para evitar confundirlo con información real. Si una verificación de permisos falla, no amplíe los privilegios del usuario de aplicación como solución rápida; identifique primero la operación y contraste su necesidad con el contrato previsto.

8. Parámetros del servicio de aplicación

El servicio central utiliza Java 21 y se conecta a la base con alertamujer_app. Desde su carpeta se copia .env.example a .env y se completan los secretos. Las tres variables de conexión identifican dirección JDBC, usuario y contraseña. JWT_SECRET protege las sesiones y requiere una longitud mínima de 32 bytes; sustituirlo invalida las sesiones emitidas previamente. CORS_ALLOWED_ORIGINS enumera los orígenes web autorizados, incluidos esquema y puerto, separados por comas. EVIDENCE_STORAGE_PATH señala una carpeta privada y persistente para el contenido fotográfico.

Los parámetros SMTP_* configuran el envío de correo usado en la verificación y recuperación de cuentas. SMTP_FROM identifica al remitente; cuando se usa un proveedor que requiere contraseña de aplicación, esa credencial se configura allí, sin exponerla en el documento. La variable IDENTITY_PHONE_SIMULATOR_ENABLED controla el simulador de verificación telefónica del flujo de registro actual. El simulador no envía mensajes SMS y solo debe describirse como tal. Antes del arranque, el responsable revisa que la contraseña del usuario de aplicación coincida con la base y que el origen web se encuentre en la lista permitida. Una configuración coherente reduce fallas de autenticación, correo y acceso desde el navegador.

La ruta de evidencia debe existir o poder ser creada por el proceso, y debe disponer de espacio suficiente para los archivos de prueba. En una ejecución con Docker se usa una ruta interna montada sobre volumen; en la ejecución local puede indicarse una ruta privada del equipo. Verifique que esa ruta no se publica como contenido web. Si el envío de correo falla, conserve la respuesta técnica y compruebe los parámetros SMTP antes de solicitar varios códigos nuevos.

9. Ejecución local del servicio

Al ejecutar el servicio directamente con Maven en Windows, el archivo .env no se carga por sí mismo. El equipo debe incorporarlo a la sesión de PowerShell que iniciará la aplicación. La dirección JDBC del ejemplo usa host.docker.internal, útil cuando el servicio corre en un contenedor; en la ejecución directa sobre el equipo se reemplaza por 127.0.0.1. El procedimiento siguiente se aplica después de completar el archivo privado y de confirmar que la base está lista.

```
Get-Content .env | ForEach-Object {
   if ($_ -match '^([A-Za-z_][A-Za-z0-9_])=(.)$') {
      Set-Item -Path "Env:$($matches[1])" -Value $matches[2]
   }
}
$env:SPRING_DATASOURCE_URL =
$env:SPRING_DATASOURCE_URL.Replace('host.docker.internal', '127.0.0.1')
.\mvnw.cmd spring-boot:run '-Dspring-boot.run.arguments=--server.port=8081'
```

El servicio debe iniciar sin errores de conexión ni de validación de estructura. La API queda bajo http://localhost:8081/api/v1, y la ruta técnica de salud se consulta en http://localhost:8081/actuator/health. Esta última combina varios indicadores; un problema de SMTP puede hacer que informe una degradación aunque las rutas REST y PostgreSQL funcionen. Por eso, ante un resultado desfavorable se examina el indicador concreto y se realiza una solicitud funcional controlada antes de concluir que todo el servicio falló.

En la primera ejecución, el envoltorio de Maven puede descargar dependencias y tardar más que en arranques posteriores. Espere a que la terminal muestre el inicio completo antes de abrir las interfaces. Si se informa un puerto ocupado, identifique el proceso que lo usa o cambie el puerto de manera coordinada en web y móvil. Si aparece una falla de validación de tablas, vuelva a la base y a Liquibase; reiniciar repetidamente el servicio no crea la estructura faltante.

10. Alternativa de ejecución del servicio con Docker

Cuando se prefiere aislar el servicio en un contenedor, se utiliza su configuración de Compose después de iniciar y migrar la base. El archivo .env del servicio mantiene los mismos datos funcionales, pero la dirección JDBC debe ser alcanzable desde el contenedor. La configuración de referencia utiliza host.docker.internal:5434 para conectarse al puerto que PostgreSQL publica en el equipo. El servicio interno escucha en el puerto 8080 del contenedor y se publica como 8081 en el equipo. La ruta de evidencia se monta sobre un volumen persistente para que las fotografías no dependan de la vida del contenedor.

```
docker compose config --quiet
docker compose up --build -d
docker compose logs backend
```

Este arranque inicia únicamente el servicio de aplicación; no reemplaza la preparación previa de PostgreSQL y Liquibase. Si se decide conectar ambos ecosistemas por una red Docker compartida, deben coincidir el nombre de red y la dirección JDBC de esa red. No debe alternarse sin revisión entre la dirección interna de Docker y la dirección del host, porque se refieren a rutas distintas. Al verificar el arranque, se examinan los registros del servicio, el puerto publicado y una solicitud de salud o autenticación. El equipo elige una sola modalidad de ejecución durante la prueba para evitar dos instancias que compitan por el mismo puerto.

El volumen de evidencia merece una comprobación específica: envíe una fotografía sintética desde una cuenta autorizada, reinicie solo el contenedor del servicio y confirme que el archivo sigue consultable. Esta prueba se realiza después de las verificaciones de identidad y alerta, no al inicio. Si el contenedor informa una ruta sin permisos de escritura, corrija el montaje o los permisos del volumen y repita el envío; no traslade las fotografías a la carpeta pública de la web.

11. Preparación de la experiencia web

La web integra un sitio público para visitantes y un panel reservado para administración. Usa React, TypeScript y Vite. Su configuración local indica la dirección base del servicio mediante VITE_API_BASE_URL, que corresponde al origen, por ejemplo http://localhost:8081; las rutas de la API se agregan internamente. VITE_WS_BASE_URL puede indicar el origen WebSocket; si se omite, se deriva de la dirección HTTP. VITE_API_TIMEOUT_MS define cuánto espera el navegador antes de informar que una solicitud tardó demasiado. La lista de orígenes permitidos del servicio debe incluir el puerto de Vite.

```
Copy-Item .env.example .env
npm install
npm run dev
```

Estos comandos se ejecutan dentro de la carpeta web. La terminal muestra la dirección local para abrirla en el navegador. Un visitante puede consultar la presentación del producto, sus funciones y contenidos de seguridad sin iniciar sesión. La persona administradora entra por la pantalla de acceso web y, tras autenticarse, ve las áreas autorizadas de tablero, alertas, usuarias y auditoría. Si el sitio carga pero una operación administrativa falla, se revisan en este orden la dirección base, el estado del servicio, las credenciales, el origen CORS y la respuesta recibida. El aspecto visual por sí solo no confirma la conexión con los datos.

La web se puede comprobar por etapas. Primero abra una página pública y confirme que la navegación funciona. Después ingrese con la cuenta administrativa y consulte un listado que provenga del servicio. Finalmente abra el detalle de una alerta de prueba y observe si aparecen el estado, la conversación y la evidencia permitida. Si una etapa falla, anote la dirección visible y el mensaje presentado; ello diferencia una ruta de navegador incorrecta de una respuesta de la API.

12. Preparación de la experiencia móvil

La aplicación móvil utiliza React Native, Expo Router y TypeScript. En su archivo .env, EXPO_PUBLIC_API_BASE_URL debe apuntar al origen del servicio que el dispositivo realmente pueda alcanzar. Para un teléfono físico se usa la dirección de red local del equipo y el puerto 8081; localhost en el teléfono se referiría al propio dispositivo. El equipo y el teléfono deben compartir una red que permita la comunicación. El emulador Android dispone de una dirección especial para referirse al equipo anfitrión, indicada por la guía técnica del servicio.

```
Copy-Item .env.example .env
npm install
npm start
```

Estos comandos se ejecutan en la carpeta móvil. npm start inicia la sesión de desarrollo de Expo y permite abrir la experiencia con Expo Go mediante el código QR mostrado. Expo Go es un medio de prueba de la aplicación durante el desarrollo. Un archivo APK, cuando se entrega, se instala como paquete Android por separado; Expo Go no instala ni distribuye ese archivo (Expo, s. f.-a, s. f.-b). Además, el propio cliente móvil distingue su ejecución en Expo Go y no exige allí la validación del servicio persistente de ubicación en segundo plano. Para comprobar esa capacidad se necesita una compilación Android apropiada y una prueba en dispositivo compatible. La guía evita presentar una sesión QR como certificación de comportamiento en segundo plano.

Antes de probar SOS en un teléfono, abra la aplicación y revise sus permisos de ubicación y cámara en los ajustes del sistema. La falta de cámara no bloquea por sí sola SOS, pero impide fotografías. La prueba en Expo Go permite recorrer varias pantallas, aunque no certifica el servicio persistente de ubicación Android. Una compilación Android destinada a esa verificación debe instalarse y probarse por separado, con el mismo origen de servicio y una cuenta sintética.

13. Contratos de comunicación y permisos

La web y el móvil consumen el mismo servicio central, pero cada uno muestra funciones acordes con su público. Las solicitudes protegidas incluyen un token de acceso; el servicio comprueba su firma, su vigencia y el estado de la sesión antes de responder. La cuenta de una usuaria accede a sus datos, contactos, alertas y contenido autorizado. La cuenta administrativa accede al panel web, al seguimiento de alertas, a la gestión de usuarias y a los registros de auditoría. Un visitante del sitio público no obtiene por ello permisos administrativos.

El intercambio ordinario usa solicitudes HTTP. Para el chat y ciertos cambios de estado de una emergencia, el servicio ofrece una conexión WebSocket con mensajes STOMP. Esa conexión pasa por el servicio y por sus controles de acceso; no conecta directamente dos teléfonos entre sí. En una ejecución local de una sola instancia, el distribuidor de mensajes mantiene canales en memoria. Las fotografías se consultan mediante rutas autorizadas; no quedan expuestas como una carpeta pública. La implementación debe respetar estas distinciones al configurar direcciones y al preparar cuentas de prueba. Si una pantalla informa acceso denegado, se confirma primero el rol y la relación con la alerta antes de tratar el resultado como un error de conectividad.

Como prueba de límites, abra una alerta con una cuenta y acceda desde otra que no sea titular, contacto aceptado ni administradora. El servicio debe negar el contenido protegido aunque se conozca su identificador. Realice esta comprobación solo con datos sintéticos y registre el código de respuesta, sin copiar tokens en el informe. Al terminar, cierre las sesiones de prueba y verifique que el acceso protegido vuelve a solicitar autenticación.

14. Recorrido de registro y sesión

La primera comprobación funcional se realiza con una cuenta de prueba creada desde el móvil. El flujo vigente comienza con nombre de usuario, nombres y teléfono; el paso telefónico utiliza un simulador controlado y muestra un código de prueba cuando está habilitado. Después se completan correo, contraseña y aceptación de condiciones, y se confirma un código enviado por correo. El equipo debe disponer de un buzón de prueba accesible y de una configuración SMTP válida. Un resultado favorable es que la cuenta se active tras ambas verificaciones y pueda entrar desde el móvil. No debe interpretarse el código simulado como un SMS real.

La persona administradora usa su cuenta inicial para entrar directamente desde la web. También puede crear cuentas de usuaria habilitadas desde el panel; esas cuentas tienen un origen distinto del registro público y no atraviesan el mismo circuito OTP. Para validar la sesión, se comprueba que cada persona vea solo su área autorizada y que cerrar sesión retire el acceso. Si el correo no llega, se revisan autenticación SMTP, remitente, bandeja de spam y el error devuelto por el servicio. Una respuesta aceptada por el proveedor de correo acredita el envío desde el servicio, pero la recepción final se confirma en el buzón de prueba.

La comprobación de recuperación de contraseña se puede añadir después del primer acceso. Solicite el código al correo de prueba, establezca una nueva contraseña y confirme que la anterior deja de funcionar. Si se reenvía un código, use el último recibido y respete la espera indicada por el sistema. Este recorrido también verifica la configuración de correo y la revocación de sesiones sin recurrir a datos de personas reales.

15. Contactos y activación de una alerta

Con la sesión de usuaria activa, la prueba agrega un contacto mediante el directorio de personas registradas. La invitación se acepta desde la cuenta destinataria y solo después se considera una relación autorizada. La aplicación muestra si el contacto cumple las condiciones para participar en una emergencia. El botón SOS exige al menos un contacto aceptado y elegible, permisos de ubicación pertinentes, GPS y conectividad. La cámara sirve para fotografías, pero su permiso ausente no debe describirse como una condición que bloquea por sí sola la activación del SOS. Estas condiciones se observan en la pantalla inicial antes de iniciar la prueba.

La activación requiere mantener presionado el control SOS y confirmar la decisión. El servicio registra la alerta y la aplicación muestra su estado y la última ubicación confirmada. Durante una alerta abierta, el dispositivo intenta mantener la información de ubicación y comunicar latidos de actividad; la exactitud y continuidad dependen de permisos, señal y condiciones del dispositivo. El contacto autorizado consulta las alertas abiertas desde su área móvil, que se actualiza periódicamente mientras está activa la aplicación. Este mecanismo no equivale a una notificación remota garantizada. Al terminar, se verifica que el estado final aparezca en el historial de la titular y en el panel administrativo.

El recorrido se documenta mejor en una línea temporal: hora de aceptación del contacto, hora de activación, primera ubicación, consulta del contacto y cierre. Las horas ayudan a distinguir un retraso de actualización de una ausencia de registro. En el panel, la persona administradora puede iniciar la atención de una alerta activa; anote el nuevo estado confirmado y no confunda esa acción con una llamada telefónica o una intervención externa. Si la conectividad cae, la pantalla puede mostrar datos previos.

16. Conversación, evidencia y atención administrativa

Durante una emergencia abierta, la usuaria y una persona administradora autorizada pueden intercambiar mensajes desde sus respectivas interfaces. El servicio persiste los mensajes y publica los eventos después de confirmar el registro; así, una indicación visual de envío debe corresponder a un dato aceptado. Si la conexión en tiempo real se interrumpe, las interfaces ofrecen recuperación del historial y muestran el estado de conexión. La prueba comprueba el mensaje en ambos extremos y evita asumir que un borrador equivale a una entrega confirmada.

La usuaria puede tomar una fotografía manual desde la alerta. El cliente la convierte a WebP y la envía a un almacenamiento privado, con límite de 500 000 bytes por archivo. La implementación actual también intenta capturas automáticas cada diez segundos mientras la alerta y la aplicación están activas en primer plano, la cámara está autorizada y preparada. El intento se pausa al pasar a segundo plano y no debe presentarse como grabación continua ni como una garantía de captura. El administrador puede consultar evidencias mediante el panel tras la autorización del servicio. Para verificar esta función se comprueba la fotografía recibida y su acceso desde las cuentas previstas; se evitan imágenes sensibles o reales durante la práctica académica.

Para evitar conclusiones excesivas, distinga tres evidencias: intento de captura en el teléfono, aceptación del archivo por el servicio y apertura posterior desde una cuenta autorizada. Solo la última secuencia completa demuestra que el archivo quedó disponible para consulta. En el chat, compruebe el orden de mensajes tras reconectar. Si una evidencia antigua no aparece, revise el estado de la alerta, la autorización y el almacenamiento antes de atribuirlo a un borrado.

17. Diagnóstico, seguimiento y evidencias de aceptación

La comprobación de extremo a extremo se documenta con hechos observados y fecha, no con casillas marcadas de antemano. Una matriz breve puede reunir paso, cuenta utilizada, resultado esperado, resultado observado y acción correctiva. Las capturas deben ocultar contraseñas, códigos de verificación y datos personales innecesarios. Los registros del servicio y de Liquibase ayudan a ubicar errores, pero deben interpretarse junto con las respuestas de la interfaz y la salud de PostgreSQL. Un problema de CORS afecta al navegador, mientras que una URL móvil incorrecta afecta al teléfono; ambos pueden producir mensajes de conexión semejantes.

Para revisar el servicio antes de una entrega técnica se dispone de la verificación Maven y de una prueba de integración contra una base migrada. La web incluye comprobación de tipos, análisis estático, compilación y pruebas de navegación; el móvil incluye comprobación de tipos y análisis estático. Estos comandos son recursos de validación, no resultados ya obtenidos por este procedimiento. El equipo registra sus resultados reales en el documento de pruebas, con la versión y el entorno utilizados. Si una comprobación falla, se conserva el mensaje de error, se corrige la causa específica y se repite únicamente el tramo afectado antes de cerrar la aceptación.

Para una revisión reproducible, asocie cada captura a una fila de la matriz y elimine duplicados que no aporten información. Anote si el recorrido usó Expo Go, emulador o una compilación Android, porque las capacidades de segundo plano difieren. El mismo cuidado se aplica al correo: registre recepción del código sin mostrarlo. Si un resultado depende de una condición externa, como red o SMTP, describa su efecto observado y la repetición efectuada después de corregirla.

18. Cierre, respaldo y recuperación del entorno

El cierre de una jornada de integración comienza por terminar las alertas de prueba, cerrar sesiones y resguardar el registro de resultados. El equipo revisa que los archivos de evidencia creados para la práctica no contengan información sensible y aplica la política institucional de conservación o eliminación. La base utiliza un volumen persistente: detener los contenedores con docker compose down conserva los datos, de modo que una nueva sesión puede partir del mismo estado. Antes de una operación que cambie la estructura o las contraseñas, conviene disponer de un respaldo coherente de la base y de la carpeta privada de evidencia.

La recuperación se decide según el origen de la falla. Una variable errónea se corrige y se reinicia el componente afectado; una migración incompleta se diagnostica con el historial de Liquibase; una credencial cambiada en una base ya existente exige reconciliar los roles. docker compose down --volumes elimina el volumen local y los datos que contiene, por lo que solo se reserva para una reconstrucción deliberada de un entorno descartable. El procedimiento termina cuando los cuatro componentes se comunican, los roles ven únicamente los datos autorizados y el equipo conserva evidencias suficientes para reproducir el resultado. Esta definición de cierre ofrece una base verificable para la evaluación académica y para el mantenimiento posterior.

En la entrega final, el equipo puede adjuntar una tabla de cierre con el estado de cada componente, la versión utilizada, el responsable de la comprobación y la ubicación de la evidencia. Esa tabla no reemplaza las pruebas; facilita localizar sus resultados. Quien retome el entorno debe saber si el volumen y las cuentas sintéticas siguen presentes. Al transferir la responsabilidad, entregue procedimientos y ubicaciones autorizadas, nunca secretos dentro de un documento compartido.

Referencias

Equipo del proyecto Alerta Mujer. (2026a). *AlertaMujer Backend* [Guía técnica del repositorio]. https://github.com/FreinierCardona/AlertaMujer_Backend

Equipo del proyecto Alerta Mujer. (2026b). *AlertaMujer Database* [Guía técnica del repositorio]. https://github.com/FreinierCardona/AlertaMujer_Database

Equipo del proyecto Alerta Mujer. (2026c). *AlertaMujer Frontend* [Guía técnica del repositorio]. https://github.com/FreinierCardona/AlertaMujer_Frontend

Expo. (s. f.-a). *Build APKs for Android emulators and devices*. https://docs.expo.dev/build-reference/apk/

Expo. (s. f.-b). *Start developing*. https://docs.expo.dev/get-started/start-developing/